KumoProxy on External Host — Production Setup Guide for BillionMail

Date: 2026-05-14 Scope: Replace the broken "direct-to-MX from DO" delivery path with KumoMTA-on-DO → KumoProxy (SOCKS5, on Hetzner/OVH) → recipient MX. Inputs: analysis.md (Approach B), agent_kumo_billion.md, test.md, current BillionMail repo state.

This is a complete, self-contained runbook — pick a host, install, plumb IPs, wire BillionMail, verify. No prior KumoMTA knowledge assumed.

---

0. Why KumoProxy (the one-paragraph answer)

KumoMTA's job is to open SMTP connections to recipient mail servers on TCP/25. DigitalOcean blocks outbound 25/465/587 on every Droplet and effectively never lifts the block. **KumoProxy** is a thin SOCKS5 server that ships with the KumoMTA package; you run it on a *different* host that allows outbound 25, and your DigitalOcean KumoMTA sends every SMTP session through it. The proxy host's IP becomes the sending IP — meaning your sender reputation lives where you control it, not on DO's blacklisted ranges. This is the pattern the KumoMTA team itself recommends (KumoMTA: Routing Messages Via Proxy Servers).

The cost is one extra small VM (~$5–8/mo for v1, scales linearly with sending IPs you add).

---

1. Topology — Final Picture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          DigitalOcean (control plane)                 │
│                                                                       │
│  BillionMail (Go + Vue + Postfix + Dovecot + Rspamd)                  │
│  ├─ public:  mail.gitdate.ink   → 159.89.33.85                        │
│  └─ private: 10.108.0.3                                               │
│                                                                       │
│  KumoMTA droplet (control plane only — no port-25 egress)             │
│  ├─ public:  email.gitdate.ink  → 192.241.130.241                     │
│  └─ private: 10.116.0.2                                               │
│                                                                       │
│       │ HTTP inject /api/inject/v1                                    │
│       │ (BillionMail → KumoMTA, over VPC ideally)                     │
│       ▼                                                               │
│  KumoMTA queue + shaping + DKIM signing + webhook emission            │
│                                                                       │
└──────────────────────────────┬────────────────────────────────────────┘
                               │
                               │ SOCKS5 (TCP, single port e.g. 5000)
                               │ TLS-wrapped, allowlisted by IP
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│              External proxy host (Hetzner / OVH / Vultr)              │
│                                                                       │
│  KumoProxy (binary: /opt/kumomta/sbin/proxy-server)                   │
│  ├─ eth0 primary  : <send-ip-1>   ← rDNS = mta1.gitdate.ink           │
```

```
│  ├─ eth0 alias 1  : <send-ip-2>   ← rDNS = mta2.gitdate.ink            │
│  └─ eth0 alias N  : <send-ip-N>   ← rDNS = mtaN.gitdate.ink            │
│                                                                        │
│  Outbound TCP/25 is OPEN (after port-25 unblock request).              │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
                              │
                              │ TCP/25 → recipient MX
                              ▼
                       gmail.com / outlook.com / ...
```

Key facts:

- KumoProxy is **stateless** — it just rewrites TCP source addresses on behalf of the MTA. No queue, no spool, no Lua. If it dies, KumoMTA queues until it comes back.
- KumoMTA gets *real* SMTP error codes back through the SOCKS5 channel (KumoMTA: socks5_proxy_server). This is why SOCKS5 is preferred over HAProxy — bounce classification stays intact.
- The DO KumoMTA droplet **never** opens an SMTP socket to the public internet. All it does on port 25/587 is talk *to the proxy*, which is on whatever port you pick (5000 in the docs).
- Sending-IP reputation = the **proxy host's** IPs. DO IPs are invisible to recipient MTAs.

---

2. Choose The Proxy Host

You need a Linux VM where outbound TCP/25 is allowed, with the option to add 2–N extra public IPs.

Provider	Port 25 default	How to unlock	Multi-IP support	Notes
Hetzner Cloud	Blocked	Support ticket after first paid invoice; typically approved in <24h with legitimate use case (Hetzner FAQ, Filippo Valle's notes)	Floating IPs (€1.19/mo each) or order primary IPv4 addresses	**Recommended for v1.** CX22 = €4/mo, German legal-grade privacy, good IP reputation.
OVH VPS	**Open by default in most DCs** (not Madrid) (OVH community)	No request needed; subject to outbound anti-spam screening	Failover IPs (€2/mo each)	Fastest start. Pick GRA (Gravelines) or RBX (Roubaix) datacenters.
Vultr High-Frequency	Blocked	Support ticket; approval rate moderate, requires justification (Vultr docs)	Reserved IPs	OK fallback.
Linode	Blocked	Ticket; recent reports of stricter enforcement	Yes	Slower than Hetzner for unblock.

Bare metal (Hetzner AX-line / OVH dedicated)	Open or unblockable	Same as VPS lines	Many	Only worth it once you outgrow >4 IPs.

Recommendation: Hetzner Cloud CX22 (2 vCPU, 4 GB, 1 IPv4) in `nbg1` or `fsn1`. The unblock ticket is well-trodden; their template even allows pre-emptive submission.

Avoid: AWS, GCP, Azure, DigitalOcean (yes, DO again — port 25 is blocked globally on their network, no exceptions for a second droplet either).

---

3. Pre-Flight Before Renting Anything

You need the right DNS already published before mail goes out, or first sends get classified as spam and you burn the IPs.

1. **Sending hostname:** you'll use one per IP, e.g. `mta1.gitdate.ink`, `mta2.gitdate.ink`. These are the EHLO names KumoMTA will announce; they must FCrDNS (forward = reverse) ([Mailflow Authority — PTR best practices](#)). Gmail/Yahoo enforce this since Feb 2024.
2. **SPF planning:** your sender domain (`email.gitdate.ink`) currently has `v=spf1 ip4:192.241.130.241 ~all`. After this rollout it will become `v=spf1 ip4:<send-ip-1> ip4:<send-ip-2> -all`. Do **not** change SPF until step 8 (the actual cutover).
3. **DKIM:** `s1._domainkey.email.gitdate.ink` already exists. KumoMTA on DO will keep signing — signing happens *before* the proxy hop, so DKIM is untouched by this work.
4. **DMARC:** `_dmarc.email.gitdate.ink` already exists per your PRD §4.4. Leave it.
5. **Hostnames to register A records for:**
   - `mta1.gitdate.ink → <send-ip-1>` (set after IP assignment)
   - `mta2.gitdate.ink → <send-ip-2>`
   - …
6. **Test mailbox somewhere outside Gmail/Outlook** (e.g. a Postmark or Mailtrap inbox) for first-mile validation, then add Gmail / Outlook / Yahoo for FCrDNS validation only after warmup.

---

4. Provision The Proxy Host (worked example: Hetzner)

Times below are real-world averages from public threads.

4.1 Order the box (15 min)

```
Hetzner Cloud → Servers → New
Image:    Ubuntu 24.04 LTS
Type:     CX22 (2 vCPU / 4 GB / 40 GB)   - €4.15/mo
Location: nbg1 (Nuremberg) or fsn1 (Falkenstein)
SSH key:  paste your ops key
Name:     kumoproxy-1
```

After provisioning, note the primary IPv4 (e.g., `78.46.x.y`) and primary IPv6 (e.g., `2a01:4f8:...`). This becomes `mta1.gitdate.ink`.

4.2 File the port-25 unblock ticket (5 min, then wait <24h)

Hetzner template ([Hetzner Cloud FAQ](#)):

```
Subject: Outbound port 25 unblock request — kumoproxy-1

We operate a transactional/marketing email platform for our SaaS
(billionmail / gitdate.ink). This Hetzner instance will run a SOCKS5
proxy in front of an external KumoMTA cluster, sending mail only on
behalf of opted-in customers under domains we own.

Compliance:
- SPF / DKIM / DMARC published on all sending domains.
- Suppression list, complaint handling, easy unsubscribe per CAN-SPAM
  and GDPR.
- Bounce rate target <3%, complaint rate target <0.1%.
- Volume start: 1k/day, target 1M/month within 90 days.

Hostname:   mta1.gitdate.ink
IPv4:       78.46.x.y
Use case:   Outbound SMTP for our own customers' transactional mail.

Please open outbound TCP/25 on this IP. Happy to discuss volume caps.
```

Submit via Hetzner Cloud → Support → New ticket → "Server / cloud → network limit / port 25". First paid invoice must already be cleared; for a brand-new CX22 you pay €4.15 upfront if you select monthly, which counts.

4.3 Set rDNS on the primary IP (2 min)

Hetzner Cloud Console → Servers → kumoproxy-1 → Networking → IP addresses → set reverse DNS:

```
78.46.x.y     →   mta1.gitdate.ink
2a01:4f8:...  →   mta1.gitdate.ink
```

Wait 1–10 min for propagation. Verify:

```
dig +short -x 78.46.x.y          # should return mta1.gitdate.ink.
dig +short mta1.gitdate.ink       # should return 78.46.x.y
```

This is the FCrDNS check. Don't skip — Gmail will hard-block without it.

4.4 Lock down the box (10 min)

```
ssh root@78.46.x.y

# Create ops user
adduser --disabled-password --gecos "" ops
usermod -aG sudo ops
mkdir /home/ops/.ssh && cp /root/.ssh/authorized_keys /home/ops/.ssh/
chown -R ops:ops /home/ops/.ssh && chmod 700 /home/ops/.ssh
```

```
# UFW: allow only SSH and SOCKS5 from your KumoMTA droplet
apt update && apt install -y ufw
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
ufw allow from 192.241.130.241 to any port 5000 proto tcp   # KumoMTA droplet only
ufw enable

# Disable root password login; require key only
sed -i 's/^#\?PermitRootLogin .*/PermitRootLogin prohibit-password/'
/etc/ssh/sshd_config
sed -i 's/^#\?PasswordAuthentication .*/PasswordAuthentication no/'
/etc/ssh/sshd_config
systemctl restart sshd
```

The `ufw allow from 192.241.130.241 to any port 5000` is the **critical security boundary** — KumoProxy has no built-in authentication, so source-IP allowlisting is your only access control. Don't expose 5000 to the internet ([KumoMTA: KumoProxy docs](#) — security model is "trust the network").

---

5. Install KumoProxy

KumoProxy ships *inside* the KumoMTA package. You install `kumomta` on the proxy box but you only run the `proxy-server` binary; the main `kumomta` daemon stays disabled.

5.1 Install the KumoMTA package (5 min)

Per [KumoMTA Linux install](#):

```
sudo apt install -y curl gnupg ca-certificates
curl -fsSL https://openrepo.kumomta.com/kumomta-ubuntu-22/public.gpg \
  | sudo gpg --yes --dearmor -o /usr/share/keyrings/kumomta.gpg
sudo chmod 644 /usr/share/keyrings/kumomta.gpg
curl -fsSL https://openrepo.kumomta.com/files/kumomta-ubuntu22.list \
  | sudo tee /etc/apt/sources.list.d/kumomta.list > /dev/null
sudo apt update && sudo apt install -y kumomta

# Make sure the main daemon is NOT running on this box
sudo systemctl disable --now kumomta
sudo systemctl disable --now kumo-tsa-daemon
```

Note: KumoProxy is only documented as tested on Ubuntu 24, but the Ubuntu 22 package works in practice; use 24.04 if you want to follow the docs verbatim.

Confirm the binary exists:

```
ls -lh /opt/kumomta/sbin/proxy-server
/opt/kumomta/sbin/proxy-server --help
```

5.2 Kernel tuning (3 min)

KumoProxy holds one TCP connection per SMTP session, so file-handle limits and ephemeral port range matter. Create `/etc/sysctl.d/99-kumoproxy.conf`:

```
# More ephemeral ports for outbound connections
net.ipv4.ip_local_port_range = 10000 65535

# Faster reuse of TIME_WAIT sockets (outbound only)
net.ipv4.tcp_tw_reuse = 1

# Allow binding to non-local IPs (needed if you ever switch to fancier routing)
net.ipv4.ip_nonlocal_bind = 1

# Forwarding for any future multi-NIC scenarios (harmless on a single-NIC box)
net.ipv4.ip_forward = 1

# Conntrack limits (set high enough for your peak concurrent sessions)
net.netfilter.nf_conntrack_max = 524288
```

Apply:

```
sudo sysctl --system
```

And lift the file-handle limit for the proxy user (it runs as root via systemd by default, but `root` has its own limits):

`/etc/security/limits.d/kumoproxy.conf`:

```
root soft nofile 1048576
root hard nofile 1048576
```

Reboot is the safest way to ensure everything sticks: `sudo reboot`.

5.3 Configure and start KumoProxy (5 min)

Create `/opt/kumomta/etc/kumoproxy.env`:

```
PROXY_IP="0.0.0.0"
PROXY_PORT="5000"
```

`0.0.0.0` is fine **because** you UFW-restricted port 5000 to your DO droplet IP. If your UFW rule isn't in place yet, set `PROXY_IP` to your private network address.

Create `/etc/systemd/system/kumoproxy.service` (from [KumoMTA: KumoProxy docs]):

```
[Unit]
Description=KumoMTA SOCKS5 Proxy service
After=syslog.target network.target

[Service]
Type=simple
```

```
Restart=always
RestartSec=2
EnvironmentFile=-/opt/kumomta/etc/kumoproxy.env
ExecStart=/opt/kumomta/sbin/proxy-server --listen ${PROXY_IP}:${PROXY_PORT}
TimeoutStopSec=10
LimitNOFILE=1048576

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now kumoproxy
sudo systemctl status kumoproxy        # should be active (running)
sudo journalctl -u kumoproxy -n 50 -f  # watch for startup errors
```

5.4 Smoke-test the proxy from the proxy host

```
# Open a SOCKS5 client to the local proxy, ask it to connect to Gmail's MX on 25
sudo apt install -y curl
curl --socks5-hostname 127.0.0.1:5000 -v telnet://aspmx.l.google.com:25
```

You should see Gmail's `220 mx.google.com ESMTP ...` banner. If you see `Connection refused` you forgot to start the proxy; if you see `Connection timed out` your provider hasn't unblocked port 25 yet (open the ticket again).

5.5 Smoke-test from the DO KumoMTA droplet

```
ssh root@192.241.130.241

# install a SOCKS5-aware tool
apt install -y curl

# Same test, but going through the network path your real KumoMTA will use
curl --socks5-hostname 78.46.x.y:5000 -v telnet://aspmx.l.google.com:25
```

You should see Gmail's banner. If `Connection refused`, the UFW rule is wrong or the proxy isn't listening on the public IP. If `Connection timed out`, your Hetzner port-25 ticket is still pending.

Critical: if this test passes, the entire egress path works. Everything below is wiring it into KumoMTA + BillionMail.

---

6. Add More Sending IPs (when ready to scale)

You don't have to do this on day 1 — start with the single primary IP. Add more once you have warmup discipline in place.

6.1 Order additional Hetzner primary IPv4 (€0.50/mo each, indefinite)

Hetzner Cloud → Networking → Primary IPs → New → IPv4 → assign to `kumoproxy-1`. Repeat for as many as you need.

6.2 Plumb the IPs on the NIC

Hetzner's Ubuntu 24 cloud image uses `cloud-init` + netplan. Create `/etc/netplan/60-secondary-ips.yaml`:

```yaml
network:
  version: 2
  ethernets:
    eth0:
      addresses:
        - 78.46.x.y/32        # primary (already there from DHCP; do not duplicate)
        - 78.46.a.b/32        # secondary IPv4
        - 78.46.c.d/32        # tertiary IPv4
```

```bash
sudo chmod 600 /etc/netplan/60-secondary-ips.yaml
sudo netplan --debug generate
sudo netplan apply
ip -4 addr show eth0    # all IPs should be listed
```

Reference: [Hostman: configure additional IP alias in Ubuntu](), [Hetzner Floating IP setup]().

6.3 Set rDNS for each new IP

Hetzner Console → Server → Networking → set reverse DNS for each:

```
78.46.a.b → mta2.gitdate.ink
78.46.c.d → mta3.gitdate.ink
```

Add forward A records under `gitdate.ink`:

```
mta1.gitdate.ink   A   78.46.x.y
mta2.gitdate.ink   A   78.46.a.b
mta3.gitdate.ink   A   78.46.c.d
```

Verify FCrDNS for each:

```bash
for ip in 78.46.x.y 78.46.a.b 78.46.c.d; do
  echo "=== $ip ==="
  ptr=$(dig +short -x $ip | head -1)
  fwd=$(dig +short $ptr | head -1)
  echo "PTR : $ptr"
  echo "FWD : $fwd"
  [ "$fwd" = "$ip" ] && echo "FCrDNS: OK" || echo "FCrDNS: MISMATCH"
done
```

6.4 Verify the proxy can bind to each IP

Run this on the proxy host (with `n` set to the IP you want to validate):

```
curl --socks5-hostname 127.0.0.1:5000 \
     --interface 78.46.a.b \
     -v telnet://aspmx.l.google.com:25
```

Hmm — `curl --interface` won't push the source-IP through the SOCKS5 layer. Better test from the *DO side* using the actual `socks5_proxy_source_address` mechanism (next section): you'll see the egress IP in the SMTP `Received:` header on a test message.

(There was an early KumoProxy bug — KumoCorp/kumomta#153 — where only the first plumbed IP was usable. Confirmed fixed in recent releases; just pin to the latest `kumomta` package and test before going live with multi-IP.)

---

7. Wire KumoMTA-on-DO To The Proxy

The schema and policy generator in your repo (`core/internal/service/database_initialization/z_tenants.go`, policy.go:32) already model `kumo_egress_sources`, `kumo_egress_pools`, `kumo_egress_pool_sources`, and `tenant_sending_profiles.egress_mode = 'external_kumoproxy'`. What's **missing** is the SOCKS5 fields on the source row and the corresponding Lua emission. You have two options:

- **Option A — schema patch** (the right long-term answer). Add `socks5_proxy_server` + `socks5_proxy_source_address` columns to `kumo_egress_sources`, expose them in the source CRUD, and emit them from `policy.go:buildEgressSourcesFile`. Fits cleanly into PRD Phase 8.
- **Option B — Lua-only for v1** (zero schema change, ship today). Reuse the existing `source_address` column to hold the public IP plumbed on the proxy host, and hardcode the proxy host:port in the generator. This is the migration path your runbook describes: "Define every sending IP as a separate `egress_source`."

Recommend Option B for the cutover sprint, Option A as the Phase 8 follow-up.

7.1 Option B — minimum-diff Lua wiring

In core/internal/service/kumo/policy.go:271, the current generator emits:

```
b.WriteString(fmt.Sprintf("    [%d] = { name = %q, source_address = %q, ehlo_domain
= %q, ... },\n", ...))
```

Replace with a richer line that includes proxy fields when the source is `external_kumoproxy`. Pseudocode shape (don't apply yet — covered in §10):

```
b.WriteString(fmt.Sprintf(
    "    [%d] = { name = %q, source_address = %q, ehlo_domain = %q, "+
    "socks5_proxy_server = %q, socks5_proxy_source_address = %q, "+
    "node_id = %d, status = %q, warmup_status = %q },\n",
    source.ID, source.Name, "" /* not used when proxying */, source.EHLODomain,
```

```
    proxyHostPort, source.SourceAddress, // <-- the public IP plumbed on proxy
    source.NodeID, source.Status, source.WarmupStatus))
```

The `proxyHostPort` comes from a new config entry in `kumo_config` (e.g. `default_socks5_proxy = "78.46.x.y:5000"`), so you can deploy multiple proxy hosts later.

The Lua consumer on KumoMTA looks like ([KumoMTA: make_egress_source](), [socks5_proxy_server]()):

```
local sources = require '/opt/kumomta/etc/policy/egress_sources'

kumo.on('get_egress_source', function(source_name)
  for _, s in pairs(sources.sources) do
    if s.name == source_name then
      return kumo.make_egress_source {
        name = s.name,
        socks5_proxy_server         = s.socks5_proxy_server,
        socks5_proxy_source_address = s.socks5_proxy_source_address,
        ehlo_domain                 = s.ehlo_domain,
      }
    end
  end
  error('unknown egress source: ' .. source_name)
end)

kumo.on('get_egress_pool', function(pool_name)
  local entries = {}
  for _, ps in ipairs(sources.pool_sources) do
    if ps.pool == pool_name and ps.status == 'active' then
      table.insert(entries, { name = ps.source, weight = ps.weight })
    end
  end
  return kumo.make_egress_pool { name = pool_name, entries = entries }
end)
```

7.2 Seed the database rows

For each plumbed proxy IP, insert one `kumo_egress_sources` row. Either via the (future) admin UI or directly:

```
-- Source per plumbed IP on the proxy
INSERT INTO kumo_egress_sources (name, source_address, ehlo_domain, node_id, status,
warmup_status)
VALUES
 ('gitdate-mta1', '78.46.x.y', 'mta1.gitdate.ink', 0, 'active', 'warm'),
 ('gitdate-mta2', '78.46.a.b', 'mta2.gitdate.ink', 0, 'active', 'warming'),
 ('gitdate-mta3', '78.46.c.d', 'mta3.gitdate.ink', 0, 'active', 'warming');

-- Pool for tenants who share these IPs
INSERT INTO kumo_egress_pools (name, description, status)
VALUES ('shared-default', 'Default shared pool on kumoproxy-1', 'active');
```

```sql
-- Bind sources to pool with weights (start lighter on warming IPs)
INSERT INTO kumo_egress_pool_sources (pool_id, source_id, weight, status)
SELECT
  (SELECT id FROM kumo_egress_pools WHERE name = 'shared-default'),
  id, CASE warmup_status WHEN 'warm' THEN 100 WHEN 'warming' THEN 25 ELSE 10 END,
  'active'
FROM kumo_egress_sources WHERE name LIKE 'gitdate-mta%';

-- Point all profiles at this pool by default
UPDATE tenants SET default_kumo_pool = 'shared-default' WHERE default_kumo_pool =
'';
UPDATE tenant_sending_profiles
SET kumo_pool_id = (SELECT id FROM kumo_egress_pools WHERE name = 'shared-default'),
    kumo_pool_name = 'shared-default',
    egress_mode = 'external_kumoproxy'
WHERE kumo_pool_id = 0;
```

7.3 Deploy the generated policy to KumoMTA

The generator writes to `policy/egress_sources.lua` (policy.go:287) as part of the preview/deploy flow:

1. Trigger config preview from BillionMail (`POST /api/kumo/config/preview` — already implemented).
2. Inspect the rendered `egress_sources.lua` in the UI under Settings → KumoMTA → Preview.
3. SCP the generated files to the DO KumoMTA droplet's `/opt/kumomta/etc/policy/` :

   ```bash
   scp egress_sources.lua dkim.lua tenants.lua \
     root@192.241.130.241:/opt/kumomta/etc/policy/
   ```

   (Once Phase 8's managed deploy lands, this becomes a single click; for now manual SCP is the documented v1 workflow per agent_kumo_billion.md §13.)
4. Reload KumoMTA on the droplet:

   ```bash
   ssh root@192.241.130.241 'kumo-cli reload'
   # or, if reload is not wired, restart:
   ssh root@192.241.130.241 'systemctl restart kumomta'
   ```

7.4 Confirm KumoMTA loaded the proxy config

```bash
ssh root@192.241.130.241
journalctl -u kumomta -n 200 | grep -Ei 'egress|socks5|source'
```

You should see one log line per source defined, no Lua errors.

---

8. End-To-End Verification

Two layers — (1) does an injected message actually leave through the proxy with the right source IP, and (2) does BillionMail see the full lifecycle.

8.1 Layer 1 — KumoMTA → proxy → recipient

Inject a test message directly to KumoMTA:

```
curl -X POST https://email.gitdate.ink/api/inject/v1 \
  -H "Authorization: Bearer $KUMO_AUTH_SECRET" \
  -H "Content-Type: application/json" \
  -d '{
    "envelope_sender": "test@email.gitdate.ink",
    "recipients": [{ "email": "your-mailbox-outside-do@gmail.com" }],
    "content": "From: test@email.gitdate.ink\r\nTo: your-mailbox-outside-
do@gmail.com\r\nSubject: KumoProxy egress smoke test\r\nMessage-ID: <smoke-
1@gitdate.ink>\r\n\r\nIf you see this, the proxy works.\r\n"
  }'
```

In your test mailbox, open the message → View Original → find the `Received:` chain. The **outermost** Received header from your infra should show:

```
Received: from mta1.gitdate.ink ([78.46.x.y])
        by mx.google.com with ESMTPS id ...
```

Two checks must pass:

1. The IP `[78.46.x.y]` is one of your **proxy host** IPs, never `192.241.130.241` (the DO droplet) or `159.89.33.85`.
2. The hostname `mta1.gitdate.ink` matches the `ehlo_domain` you configured on the source. FCrDNS must match.

If you see DO IPs in the Received chain, the SOCKS5 wiring isn't active — re-check §7.

8.2 Layer 2 — full BillionMail lifecycle

From test.md §7-8 (already in your repo), but adapted:

```
# Send through BillionMail's API → KumoMTA → proxy → MX
curl -k -X POST https://mail.gitdate.ink/api/batch_mail/api/send \
  -H 'Content-Type: application/json' \
  -H 'X-API-Key: <api-key-for-test-tenant>' \
  -d '{ "recipient": "your-mailbox-outside-do@gmail.com" }'
```

Expected response shape:

```
{
  "accepted": true,
  "status": "queued",
  "engine": "kumomta",
  "injection_status": "pending"
}
```

Then watch the Send API log row in `api_mail_logs`:

1. `injection_status` moves `pending → injecting → queued` (KumoMTA accepted the HTTP injection).
2. After a few seconds, `delivery_status` moves to `delivered` (KumoMTA fired the webhook back to `/api/kumo/events`).
3. In your inbox, the Received header proves the proxy egress IP.

If `injection_status` stays `pending`: BillionMail → KumoMTA HTTP injection is broken — see test.md §1. If `injection_status` reaches `queued` but `delivery_status` never updates: KumoMTA can't reach the proxy, or the proxy can't reach the MX. Check `journalctl -u kumomta` on DO and `journalctl -u kumoproxy` on Hetzner. If `delivery_status = bounced` immediately: check the recipient response in the log — likely an SPF/DKIM/PTR mismatch you can fix in DNS without touching infra.

8.3 What "well integrated with BillionMail" looks like

Use this checklist (also lifted from agent_kumo_billion.md §18):

Check	Where to look	Pass
`/api/kumo/test_connection` returns `reachable`	Settings → KumoMTA → Test button	✅
`/api/kumo/status` shows last successful metrics scrape <60s old	Settings → KumoMTA → Status panel	✅
`kumo_egress_sources` has ≥1 row, status `active`	DB or Settings → KumoMTA → Sources	✅
`kumo_egress_pools` has the pool referenced by tenant default	DB or UI	✅
`tenant_sending_profiles.egress_mode = 'external_kumoproxy'` for active tenants	DB	✅
Preview config emits `socks5_proxy_server` lines for each source	Settings → KumoMTA → Preview config	✅
Send API test message reaches inbox	Test mailbox	✅
Received header shows proxy IP, not DO IP	Headers of received message	✅
`delivery_status` updates to `delivered` via webhook	`api_mail_logs` table	✅
Bounce against a known-bad address updates `delivery_status = bounced` and adds to suppression	Test against `bounce-test@gmail.com` (Gmail bounces it)	✅

When all ten pass, the integration is complete for the proxy path.

---

9. IP Warmup (Don't Skip This)

A "new" IP sending more than a few hundred messages per day to Gmail/Outlook on day 1 will be flagged. KumoMTA's TSA daemon supports automatic ramp (KumoMTA: Clustered Traffic Shaping Automation).

Recommended warmup curve (per IP, per major receiver):

Day	Daily volume	Notes
1	50	All to engaged opt-ins. Watch bounce/complaint rate.
2	100	If <2% bounce and <0.1% complaint, continue.
3	200	
4–7	500 → 2,000	Double per day.
8–14	2k → 10k	
15–30	10k → 50k	
Day 30+	full volume	Now you have reputation.

In BillionMail, set `kumo_egress_sources.warmup_status = 'warming'` and bind the source into the pool with low `weight` (e.g., 10) until day 14, then bump to 100. Your sending_profiles service already has the `warmup_enabled` flag wired (sending_profiles.go:67).

---

10. Implementation Diff Required In BillionMail

These are the minimum code changes to make the SOCKS5 path real. None of this requires a schema migration if you take Option B from §7.

10.1 Add proxy address to public Kumo config

`core/internal/service/kumo/config.go` — add `DefaultSocks5ProxyServer string` to the public config struct, render in `kumo_config` row, expose via `GET /api/kumo/config` and `POST /api/kumo/config`. The frontend (core/frontend/src/views/settings/kumo/index.vue) already has the form layout — add one more input field.

10.2 Emit `socks5_proxy_server` / `socks5_proxy_source_address` in `egress_sources.lua`

`core/internal/service/kumo/policy.go` around line 271:

```
proxy := state.Config.DefaultSocks5ProxyServer  // e.g. "78.46.x.y:5000"
for _, source := range state.Sources {
    sproxy, srcaddr := "", source.SourceAddress
    if source.NodeID == 0 && proxy != "" {
        // SOCKS5 mode: source.SourceAddress is the public IP plumbed on the proxy
        sproxy = proxy
    } else {
        srcaddr = source.SourceAddress
    }
    b.WriteString(fmt.Sprintf(
        "    [%d] = { name = %q, source_address = %q, ehlo_domain = %q, "+
        "socks5_proxy_server = %q, socks5_proxy_source_address = %q, "+
        "node_id = %d, status = %q, warmup_status = %q },\n",
        source.ID, source.Name, srcaddr, source.EHLODomain,
        sproxy, source.SourceAddress,
        source.NodeID, source.Status, source.WarmupStatus,
```

```
    ))
}
```

(`node_id == 0` means "no dedicated remote KumoMTA node, this is a proxy-fronted source.")

10.3 Add the KumoMTA-side Lua consumer

Drop a new file at `core/internal/service/kumo/templates/egress_loader.lua` (or wherever the existing Lua templates live) with the `get_egress_source` / `get_egress_pool` handlers from §7.1. The preview workflow already SCPs this to KumoMTA; just include it in `buildPreviewFiles`.

10.4 Validate at config-save time

When saving `kumo_config` with `default_socks5_proxy_server` set, run a quick TCP-reachability probe (`net.DialTimeout("tcp", server, 3*time.Second)`) and fail the save if it doesn't connect. This prevents shipping a broken preview.

10.5 Surface the path in the Sources admin UI

The frontend Sources panel (core/frontend/src/views/settings/kumo/index.vue) currently shows `source_address` / `ehlo_domain`. Add a read-only "Egress via proxy: 78.46.x.y:5000" line per source so operators see the routing at a glance.

10.6 (Optional, Phase 8) Schema columns for multi-proxy

Once you have >1 proxy host (different regions, different reputations), promote to:

```sql
ALTER TABLE kumo_egress_sources
  ADD COLUMN socks5_proxy_server VARCHAR(128) NOT NULL DEFAULT '',
  ADD COLUMN socks5_proxy_source_address VARCHAR(64) NOT NULL DEFAULT '';
```

and stop reading the global default from config. This is the clean Phase 8 shape, not needed for v1.

---

11. Operational Runbook (the boring but vital part)

11.1 Monitoring

Add these to your existing Prometheus stack (KumoMTA already exposes `/metrics`):

Metric	Alert when
`proxy reachable` (custom probe: `dial tcp 78.46.x.y:5000`)	unreachable for >2 min
Outbound `Connection timed out` rate per source	>5% over 5 min
Outbound `4xx`/`5xx` rate per recipient domain	>10% for Gmail or Outlook
Bounce rate per sending profile	>3% over 1 hour
Complaint rate per sending profile	>0.1% over 24 hours
KumoProxy systemd state	not active

Hetzner instance status (via Hetzner API)	not running

KumoProxy itself does not export Prometheus metrics — the visibility comes from KumoMTA's metrics on the DO side (per-source success/fail counts) plus the systemd unit state on the proxy host.

11.2 Failure modes and recovery

Failure	Symptom	Fix
KumoProxy crashes	KumoMTA logs `connection refused` to `78.46.x.y:5000`	`systemd Restart=always` recovers in <2s. If repeated, `journalctl -u kumoproxy` for OOM/panic.
Proxy host network outage	All sends timeout from one source	KumoMTA retries with backoff (built-in). If >5 min, manually disable the sources via `UPDATE kumo_egress_sources SET status = 'disabled'` and redeploy policy.
Hetzner re-blocks port 25 (rare)	Sudden mass timeout to MX:25	Outbound test from proxy host (`curl telnet://aspmx.l.google.com:25`). If blocked, open new Hetzner ticket. Failover plan: spin up backup proxy on OVH.
IP gets listed on Spamhaus	High deferral rate from major receivers; check Spamhaus/SURBL	Mark source `warmup_status = 'cold'`, drop weight to 1, find the source of bad mail, request delisting after fixing.
BillionMail webhook receiver down	KumoMTA accumulates pending webhooks (configured retry)	Webhook delivery resumes on its own; check `/api/kumo/events` reachability.
Lua deploy borks KumoMTA reload	KumoMTA logs Lua errors, sends pause	Keep last-known-good `egress_sources.lua` in source control on the proxy host's git mirror; SCP back and reload.

11.3 Backup proxy strategy

Once stable, run a second proxy on OVH (different region, different ASN). Add its IPs as additional `kumo_egress_sources` rows with a different `kumo_pool` (`shared-backup`). Mark tenants' fallback pool to this in `tenant_sending_profiles` . If Hetzner has a regional outage, BillionMail's `OutboundMailer` retries on the secondary pool.

11.4 Cost projection

```
Hetzner CX22                        €4.15 / mo
Additional primary IPv4 (x3)        €1.50 / mo each = €4.50
Hetzner backup snapshot             €0.40 / mo
Total v1 (1 proxy, 4 IPs)         ≈ €9 / mo
```

Compare to SES at 10M emails/month: ~$1,000. Compare to a dedicated SES IP: $24.95/mo each. KumoProxy economics dominate at >50k emails/month.

---

12. Step-By-Step Checklist (the "do this in order" version)

For the operator doing the cutover. Tick each one:

- [ ] **D-3:** Confirm DNS — `dig +short A email.gitdate.ink`, `dig +short MX gitdate.ink`, DKIM, DMARC all present per agent_kumo_billion.md §4.4.
- [ ] **D-3:** Decide proxy host provider (recommend Hetzner CX22 in `nbg1`).
- [ ] **D-3:** Pre-write the port-25 unblock ticket text (§4.2).
- [ ] **D-2:** Order Hetzner CX22, pay first invoice immediately.
- [ ] **D-2:** Submit port-25 unblock ticket. Wait for "approved" reply (usually <24h).
- [ ] **D-1:** Set primary-IP rDNS to `mta1.gitdate.ink`. Verify FCrDNS with `dig`.
- [ ] **D-1:** Add `mta1.gitdate.ink A <ip>` to your DNS.
- [ ] **D0 09:00:** Install KumoMTA package on proxy host (§5.1). Confirm `proxy-server --help` works.
- [ ] **D0 09:15:** Apply sysctl + limits (§5.2). Reboot.
- [ ] **D0 09:30:** Write `kumoproxy.env` and systemd unit (§5.3). `systemctl enable --now kumoproxy`.
- [ ] **D0 09:45:** Smoke test from proxy host (§5.4). Must see Gmail banner.
- [ ] **D0 10:00:** Lock UFW (§4.4) — restrict 5000/tcp to DO droplet's public IP.
- [ ] **D0 10:15:** Smoke test from DO droplet (§5.5). Must see Gmail banner.
- [ ] **D0 11:00:** Apply BillionMail code diff (§10.1–10.5). `go vet`, `pnpm run lint`, `go test -count=1 -short ./internal/service/...`.
- [ ] **D0 11:30:** Insert one `kumo_egress_sources` row (`gitdate-mta1`), one pool (`shared-default`), one pool-source link (§7.2).
- [ ] **D0 11:45:** Set `tenants.default_kumo_pool = 'shared-default'` for your test tenant.
- [ ] **D0 12:00:** Trigger config preview via `/api/kumo/config/preview`. Inspect `egress_sources.lua` — verify `socks5_proxy_server` is in the output.
- [ ] **D0 12:15:** SCP rendered policy files to DO KumoMTA `/opt/kumomta/etc/policy/`. Reload (`systemctl restart kumomta`).
- [ ] **D0 12:30:** Send Layer 1 test via direct HTTP injection (§8.1). Open the received message. Verify Received-from IP = `<your-proxy-ip>` (not DO IP).
- [ ] **D0 12:45:** Send Layer 2 test via BillionMail Send API (§8.2). Verify `delivery_status` reaches `delivered` via webhook.
- [ ] **D0 13:00:** Update SPF on `email.gitdate.ink` to **include the proxy IP**: `v=spf1 ip4:<proxy-ip-1> -all`. Verify with `dig +short TXT email.gitdate.ink`.
- [ ] **D0 13:30:** Soak — send 50 messages to internal test addresses, mixed Gmail / Outlook / Yahoo / your own domains. Watch for bounces.
- [ ] **D+1 to D+30:** Follow warmup ramp (§9). Add IPs only after IP-1 is steady.

---

13. Common Gotchas (from public KumoMTA / port-25 threads)

1. **Forgetting `socks5_proxy_source_address`.** Without it the proxy will still work but will use *whichever IP the kernel picks* as the source. With multiple IPs plumbed, you cannot control reputation per pool. Always set both proxy params.
2. **Setting `source_address` AND `socks5_proxy_server` on the same source.** `source_address` is for direct sending (binds locally on the MTA). Proxying ignores it. Don't set

both — it's ambiguous and confusing to read in logs. (KumoMTA: make_egress_source)

3. **Hetzner adds 24–48h "soft block" on new boxes anyway.** Even after the formal port-25 unblock, a brand-new server may have a temporary screening hold. Don't panic if Day 0 12:00 tests fail — retest at 18:00.
4. **rDNS propagation delay.** Hetzner says "instant," real-world is 1–10 min. Don't send before `dig -x` confirms.
5. **DKIM signing happens on KumoMTA (the DO side), not the proxy.** Don't try to also DKIM-sign on the proxy host. Confirm by checking the message body has exactly one `DKIM-Signature` header.
6. **SPF must include the *proxy's* IP, not DO's.** This is the SPF rewrite step in §12. Failing this is the #1 reason "everything looks fine but Gmail sends to spam" right after cutover.
7. **MX rollup and shared IPs.** If you ever set `mx_rollup = false` in shaping.toml for a specific MX, traffic to that MX won't be batched. Default behavior is fine; only change when you understand it (KumoMTA: shaping).
8. **Don't run KumoMTA daemon on the proxy host.** `systemctl disable --now kumomta` is in §5.1 for a reason — if both daemons start, they'll fight for port 25 and ports 2025/2026/etc.
9. **DigitalOcean detects sustained port-25 tunnels.** If you ever consider routing via WireGuard from DO to a port-25 host (Approach G in analysis.md), don't — DO can fingerprint sustained SMTP-shaped tunneled traffic and will suspend the droplet. SOCKS5 is fine because the DO-side TCP is on port 5000, not 25.

---

14. References

KumoMTA official

- Routing Messages Via Proxy Servers — overall topology, SOCKS5 vs HAProxy
- KumoProxy SOCKS5 Server — install, systemd, sysctl
- make_egress_source — Lua reference
- socks5_proxy_server — proxy address param
- socks5_proxy_source_address — source-IP param
- make_egress_pool — weighted pools
- Configuring Sending IPs — sources.lua / TOML helper
- Linux Installation — apt / dnf repos
- Deployment Architecture — multi-node patterns
- Clustered Traffic Shaping Automation — automated warmup
- KumoMTA GA announcement — recommended deployment shape

Hosting providers

- Hetzner Cloud FAQ — port 25
- Hetzner Floating IPs
- Hetzner Floating IP — persistent config
- Bypassing Hetzner port 25 block — Jan van den Berg
- Hetzner & email server walkthrough — Filippo Valle
- OVH community: port 25 on VPS
- Vultr: why SMTP is blocked
- DigitalOcean: why is SMTP blocked
- Captain DNS: port 25 blocked, by provider

Networking / DNS

- Configure additional IP alias in Ubuntu (Netplan)
- nixCraft: secondary IP with Netplan

- Mailflow Authority: PTR / rDNS best practices
- Mailtrap: PTR records and email sending
- MX Toolbox: SMTP Reverse DNS Resolution

Known issues

- KumoCorp/kumomta#153 — KumoProxy first-IP-only (resolved)
- KumoCorp/kumomta#45 — original KumoProxy SOCKS5 RFE

BillionMail repo anchors

- analysis.md — the broader strategy decision (this doc executes Approach B)
- agent_kumo_billion.md — phase-gated runbook
- test.md — smoke-test commands
- core/internal/service/kumo/policy.go — Lua policy generator
- core/internal/service/kumo/config.go — public config schema
- core/internal/service/sending_profiles/sending_profiles.go — tenant profile wiring
- core/internal/service/database_initialization/z_tenants.go — `kumo_egress_*` schema
- core/frontend/src/views/settings/kumo/index.vue — Kumo settings UI